C++ Programming

A Practical Guide

From first program to modern C++20 and C++23

19 chapters · 40 pages

Contents

1.	Introduction to C++	3
2.	Toolchain and Your First Program	5
3.	Types, Variables and Operators	7
4.	Control Flow	9
5.	Functions	11
6.	Arrays, Strings and Vectors	13
7.	Pointers and References	15
8.	Dynamic Memory and RAII	17
9.	Classes and Objects	19
10.	Constructors, Destructors and the Rule of Zero	21
11.	Inheritance and Polymorphism	23
12.	Operator Overloading	25
13.	Templates and Generic Programming	27
14.	STL Containers	29
15.	Iterators, Algorithms and Lambdas	31
16.	Smart Pointers and Move Semantics	33
17.	Exceptions and Error Handling	35
18.	Concurrency and Multithreading	37
19.	Modern C++ Features and Best Practices	39

Chapter 1

Introduction to C++

C++ is a general-purpose, compiled programming language that combines low-level control over memory and hardware with high-level abstractions such as classes, templates and generic algorithms. It is used wherever performance and predictability matter: operating systems, game engines, browsers, databases, embedded firmware, trading systems, scientific simulations and the runtimes of many other languages.

A short history

C++ was created by Bjarne Stroustrup at Bell Labs, beginning in 1979 under the name "C with Classes". His goal was to add the program-organisation features of Simula to the speed and portability of C. The language was renamed C++ in 1983, a pun on the C increment operator meaning "one more than C". The first commercial release came in 1985, together with Stroustrup's book The C++ Programming Language.

The language was standardised by ISO in 1998 (C++98), with a bug-fix revision in 2003. A long gap followed until C++11, often called "modern C++", which introduced move semantics, lambdas, `auto`, smart pointers, a memory model and threads. Since then the committee has shipped a new standard every three years: C++14, C++17, C++20 and C++23, with C++26 in progress.

Standard	Notable additions
C++98/03	Templates, STL, exceptions, namespaces
C++11	auto, lambdas, move semantics, smart pointers, threads
C++14	Generic lambdas, make_unique, relaxed constexpr
C++17	Structured bindings, optional, variant, filesystem
C++20	Concepts, ranges, coroutines, modules, three-way compare
C++23	std::expected, std::print, deducing this, more ranges

Design philosophy

Two principles shape almost every design decision in C++. The first is the zero-overhead principle: you do not pay for what you do not use, and what you do use is as efficient as anything you could reasonably write by hand. A `std::vector` is as fast as a manually managed array; a virtual function call costs no more than a function pointer. The second is direct mapping to hardware: the basic types and operations correspond closely to what the machine actually does, so programmers can reason about cost.

C++ is also multi-paradigm. You can write procedural code like C, object-oriented code with classes and inheritance, generic code with templates, and functional-style code with lambdas and algorithms, often mixing all four in a single program. This flexibility is a strength, but it also means

the language is large, and good C++ depends on choosing the right subset for the problem.

C++ compared with C and with managed languages

C++ is almost a superset of C, so most C code compiles as C++ with minor changes. What C++ adds is the ability to build abstractions that clean up after themselves. In C, every allocated resource must be freed manually on every path out of a function. In C++, an object's destructor runs automatically when it goes out of scope, which makes resource leaks much rarer.

Compared with Java, C# or Go, C++ has no garbage collector and no mandatory runtime. Memory is reclaimed deterministically, at a point the programmer can predict. The price is that the programmer is responsible for object lifetimes, and mistakes such as using a pointer after the object is destroyed lead to undefined behaviour rather than a clean exception. Modern C++ style, with RAII and smart pointers, removes most of this risk.

What this guide covers

The chapters that follow move from the basics of the toolchain and syntax, through functions, memory and classes, to templates, the Standard Library, error handling, concurrency and the features of the latest standards. Every chapter includes short, complete code examples that compile with any recent GCC, Clang or MSVC using at least `-std=c++20`.

Chapter 2

Toolchain and Your First Program

C++ source code is not run directly. It is translated into machine code by a compiler, combined with libraries by a linker, and the result is a native executable. Understanding this pipeline helps explain many error messages that beginners find confusing.

The compilation pipeline

Building a program happens in four stages. The preprocessor handles lines beginning with `#`, such as `#include` and `#define`, producing a single translation unit of pure C++. The compiler turns each translation unit into assembly and then into an object file (`.o` or `.obj`). Finally, the linker combines all object files and libraries into one executable, resolving references between them. A "compiler error" means the code is invalid C++; an "undefined reference" error comes from the linker and usually means a function was declared but never defined, or a library was not linked.

Choosing a compiler

The three major compilers are GCC (`g++`), Clang (`clang++`) and Microsoft Visual C++ (`cl`). On Linux, install `g++` or `clang++` from the package manager. On macOS, installing the Xcode command-line tools provides Clang. On Windows, Visual Studio Community includes MSVC, and MSYS2 provides GCC. Any of them will do for this guide.

Hello, world

```
#include <iostream>

int main() {
    std::cout << "Hello, world!\n";
    return 0;
}
```

`#include <iostream>` brings in the declarations for console input and output. `main` is the entry point; its `int` return value is reported to the operating system, where zero means success. `std::cout` is the standard output stream, and `<<` inserts a value into it. The `std::` prefix says that `cout` lives in the standard namespace.

Compile and run it from a terminal:

```
g++ -std=c++20 -Wall -Wextra -O2 hello.cpp -o hello
./hello
```

The `-std=c++20` flag selects the language standard. `-Wall -Wextra` turn on useful warnings, which you should treat as errors to be fixed. `-O2` enables optimisation. During development, add `-g` for debug symbols and `-fsanitize=address,undefined` to catch memory errors and undefined behaviour at run time.

Reading input

```
#include <iostream>
#include <string>
```

```
int main() {
    std::string name;
    int age = 0;
    std::cout << "Name and age: ";
    std::cin >> name >> age;
    std::cout << name << " will be " << age + 1 << " next year.\n";
}
```

`std::cin >> x` reads whitespace-separated values. To read a whole line, including spaces, use `std::getline(std::cin, line)`. Note that `main` may omit `return 0;`; reaching the end of `main` returns zero automatically.

Build systems

Real projects contain many files, so nobody types compiler commands by hand. CMake is the de facto standard build generator. A minimal `CMakeLists.txt` declares the minimum CMake version, a project name, the C++ standard, and an executable built from a list of sources. Running `cmake -S . -B build` followed by `cmake --build build` then produces the program on any platform. Package managers such as vcpkg and Conan integrate with CMake to fetch third-party libraries.

Chapter 3

Types, Variables and Operators

C++ is statically typed: every variable has a type fixed at compile time, and the compiler checks that operations are valid for that type. This catches many mistakes before the program ever runs and lets the compiler generate efficient code.

Fundamental types

Type	Typical size	Purpose
bool	1 byte	true or false
char	1 byte	A character or small integer
int	4 bytes	Default integer type
long long	8 bytes	Large integers
float	4 bytes	Single-precision floating point
double	8 bytes	Default floating-point type
std::size_t	8 bytes (64-bit)	Sizes and indices

Sizes are platform-dependent; only minimums are guaranteed. When an exact width matters, include `<cstdint>` and use `std::int32_t`, `std::uint64_t` and friends. Unsigned types wrap around on overflow, while signed overflow is undefined behaviour, so prefer signed types for arithmetic and reserve unsigned types for bit manipulation.

Declaring and initialising variables

```
int count = 10;          // copy initialisation
double ratio{0.75};      // brace (uniform) initialisation
auto total = count * 2;  // type deduced as int
const double pi = 3.14159;
constexpr int max_users = 100;  // known at compile time

int narrow{3.7};  // error: brace init forbids narrowing
```

Brace initialisation is preferred in modern code because it refuses narrowing conversions such as silently truncating a `double` to an `int`. `auto` lets the compiler deduce the type from the initialiser, which reduces repetition and avoids accidental conversions. `const` marks a value as read-only; `constexpr` additionally requires that it be computable at compile time, so it can be used for array sizes and template arguments.

Always initialise variables. A local `int x;` without an initialiser holds an indeterminate value, and reading it is undefined behaviour.

Operators

C++ provides the familiar arithmetic operators `+ - * / %`, comparison operators `== != < <= > >=`, logical operators `&& || !`, and bitwise operators `& | ^ ~ << >>`. Integer division truncates toward zero, so `7 / 2` is `3`; convert one operand to `double` if you need `3.5`. The compound

assignments `+=`, `-=` and so on modify a variable in place, and `++i` and `i++` increment it, the prefix form returning the new value and the postfix form the old one.

C++20 adds the three-way comparison operator `<=>`, nicknamed the spaceship operator, which returns whether one value is less than, equal to, or greater than another in a single call. Defining it for a class gives you all six comparison operators at once.

Type conversions

```
double d = 9.99;
int i = static_cast<int>(d);      // explicit, i == 9
long big = 1'000'000'000L;        // digit separators
unsigned u = static_cast<unsigned>(-1); // wraps to max value
```

Implicit conversions happen silently between arithmetic types, which can lose information. When you mean a conversion, write it with `static_cast`. The C-style cast `(int)d` still compiles but is discouraged, because it can perform dangerous reinterpretations without warning. The other named casts, `const_cast`, `reinterpret_cast` and `dynamic_cast`, have narrow, specialised uses covered in later chapters.

Scope and lifetime

A variable declared inside braces exists only until the closing brace; this is its scope. Variables declared outside any function have static storage duration and live for the whole program, but global mutable state makes code hard to reason about and test, so keep it to a minimum. Names can be grouped into namespaces, such as `namespace geometry { ... }`, to avoid collisions between libraries.

Chapter 4

Control Flow

Control-flow statements decide which code runs and how many times. C++ inherits its control structures from C and adds a few conveniences, such as range-based loops and initialisers inside `if` and `switch`.

if and else

```
int score = 72;
if (score >= 90) {
    std::cout << "A\n";
} else if (score >= 70) {
    std::cout << "B or C\n";
} else {
    std::cout << "Needs work\n";
}

// C++17: if with initialiser keeps 'it' scoped to the statement
if (auto it = table.find(key); it != table.end()) {
    use(it->second);
}
```

The condition must be convertible to `bool`. Integers and pointers convert implicitly, with zero and `nullptr` becoming `false`. Always use braces, even for single statements; it prevents a whole class of bugs when lines are added later. The ternary operator `cond ? a : b` is a compact expression form useful for simple choices.

switch

```
enum class Color { Red, Green, Blue };

std::string name(Color c) {
    switch (c) {
        case Color::Red:   return "red";
        case Color::Green: return "green";
        case Color::Blue:  return "blue";
    }
    return "unknown";
}
```

`switch` works on integers and enumerations, not strings. Each `case` falls through to the next unless it ends with `break` or `return`; if fall-through is intentional, mark it with the `[[fallthrough]]` attribute so readers and the compiler know. Compilers warn when a switch over an enum misses a value, which is a good reason to avoid a `default` label when all values are handled explicitly.

Prefer `enum class` over plain `enum`. Its values are scoped (`Color::Red` instead of `Red`) and do not implicitly convert to integers, which prevents accidental comparisons between unrelated enumerations.

Loops

```
for (int i = 0; i < 5; ++i) {
    std::cout << i << ' ';
}

std::vector<int> v{3, 1, 4, 1, 5};
for (int x : v) { std::cout << x << ' '; }        // range-based for
for (int& x : v) { x *= 2; }                       // modify in place
for (const auto& s : names) { std::cout << s; }   // no copies

int n = 10;
while (n > 0) { n /= 2; }

do {
    std::cout << "runs at least once\n";
} while (false);
```

The classic `for` loop has an initialiser, a condition and an increment. The range-based `for` loop, introduced in C++11, iterates over every element of a container, array or any type that provides `begin()` and `end()`. Use `const auto&` for elements you only read, to avoid copying, and `auto&` for elements you modify. `while` checks its condition before each iteration, `do ... while` after, so its body always runs at least once.

break, continue and goto

`break` exits the innermost loop or switch, while `continue` skips to the next iteration. To leave several nested loops at once, move them into a function and `return`, which is clearer than the rarely justified `goto`. Loops that should never end are written `while (true)` or `for (;;)`, with a `break` or `return` inside.

Common pitfalls

Writing `if (x = 5)` assigns instead of comparing; compilers warn about this, which is another reason to enable warnings. Comparing a signed loop counter with an unsigned size such as `v.size()` also triggers warnings; use `std::size_t` for indices, or better, a range-based loop. Finally, modifying a container's size while looping over it with iterators or a range-based loop invalidates the iteration and leads to undefined behaviour.

Chapter 5

Functions

Functions are the basic unit of reuse. A function has a return type, a name, a parameter list and a body. In C++ a function must be declared before it is called; the declaration (prototype) is usually placed in a header file and the definition in a source file.

Declaring and defining functions

```
// math_utils.h
#pragma once
int add(int a, int b);
double average(const std::vector<double>& values);

// math_utils.cpp
#include "math_utils.h"
int add(int a, int b) { return a + b; }

double average(const std::vector<double>& values) {
    if (values.empty()) return 0.0;
    double sum = 0.0;
    for (double v : values) sum += v;
    return sum / values.size();
}
```

`#pragma once` prevents a header from being included twice in the same translation unit. Each function must be defined exactly once across the whole program; this is the One Definition Rule. Small functions defined in headers must be marked `inline` (or be templates, or member functions defined in the class body) to avoid multiple-definition linker errors.

Passing parameters

How a parameter is passed matters for both correctness and performance. Pass by value (`int x`) for cheap types such as numbers and small structs; the function gets its own copy. Pass by const reference (`const std::string& s`) for larger objects you only need to read, which avoids a copy. Pass by non-const reference (`std::string& out`) when the function must modify the caller's object, though returning a value is usually clearer. Pass a pointer when "no object" is a valid input, represented by `nullptr`.

```
void increment(int& n) { ++n; }
void print(const std::string& s) { std::cout << s << '\n'; }

int x = 1;
increment(x);   // x is now 2
```

Default arguments and overloading

```
void log(const std::string& msg, int level = 1);

int    square(int x)    { return x * x; }
double square(double x) { return x * x; }

square(3);    // calls int version
```

```
square(2.5);  // calls double version
```

Default arguments let callers omit trailing parameters. Overloading allows several functions with the same name but different parameter types; the compiler chooses the best match at compile time. Overloads that differ only in return type are not allowed, and ambiguous calls are compile errors.

Returning values

Return results by value. Modern compilers apply return value optimisation, constructing the result directly in the caller's storage, and C++17 guarantees this for returned temporaries, so returning even a large vector is cheap. To return several values, return a struct or use `std::pair`/`std::tuple` with structured bindings: `auto [quotient, remainder] = divide(17, 5);`. Never return a reference or pointer to a local variable; the local is destroyed when the function returns, leaving a dangling reference.

constexpr and inline functions

```
constexpr long long factorial(int n) {
    return n <= 1 ? 1 : n * factorial(n - 1);
}
static_assert(factorial(5) == 120);  // evaluated at compile time
```

A `constexpr` function can run at compile time when given constant arguments, and at run time otherwise. C++20 adds `consteval` for functions that must run at compile time. Recursion is fully supported, but each call uses stack space, so deep recursion on large inputs can overflow the stack; iterative solutions are safer in that case.

Chapter 6

Arrays, Strings and Vectors

Most programs work with collections of values. C++ offers raw C-style arrays for compatibility, but modern code almost always uses the Standard Library types `std::array`, `std::vector` and `std::string`, which know their own size and manage their own memory.

C-style arrays and std::array

```
int raw[5] = {1, 2, 3, 4, 5};   // size fixed, no bounds info
std::array<int, 5> arr{1, 2, 3, 4, 5};

arr.size();     // 5
arr[2];         // 3, unchecked
arr.at(10);     // throws std::out_of_range
```

A raw array decays to a pointer to its first element when passed to a function, losing its size, which is a frequent source of buffer overruns. `std::array<T, N>` wraps a fixed-size array with no overhead but keeps the size, supports copying and comparison, and works with every standard algorithm. Use it when the number of elements is known at compile time.

std::vector: the default container

```
#include <vector>

std::vector<int> v;          // empty
v.push_back(10);
v.push_back(20);
v.emplace_back(30);          // constructs in place
std::vector<int> w(5, 0);    // five zeros

for (std::size_t i = 0; i < v.size(); ++i) std::cout << v[i];
v.pop_back();
v.insert(v.begin(), 5);      // O(n): shifts elements
v.clear();
```

`std::vector` is a dynamic array: elements are stored contiguously, it grows as needed, and it frees its memory automatically. Access by index is constant time, appending at the end is amortised constant time, and inserting or erasing in the middle is linear because later elements must shift. Because of contiguous storage and cache friendliness, vector is the right default container in the vast majority of cases.

When a vector runs out of capacity it allocates a larger block, typically 1.5 to 2 times bigger, and moves its elements. Calling `reserve(n)` beforehand avoids repeated reallocations when you know roughly how many elements will be added. Reallocation invalidates all pointers, references and iterators into the vector, so do not hold on to them across a `push_back`.

std::string

```
#include <string>

std::string greeting = "Hello";
```

```
greeting += ", world";
greeting.size();                // 12
greeting.substr(0, 5);          // "Hello"
greeting.find("world");         // 7, or std::string::npos
greeting.replace(0, 5, "Howdy");
std::string num = std::to_string(42);
int n = std::stoi("123");
```

`std::string` is a mutable, owning sequence of characters that manages its own memory. It supports concatenation with `+` and `+=`, comparison with `==` and `<`, searching with `find`, and extraction with `substr`. Many implementations use small-string optimisation, storing short strings inside the object itself without any heap allocation.

std::string_view

C++17's `std::string_view` is a non-owning, read-only view of a sequence of characters: just a pointer and a length. It is ideal as a function parameter when you only need to read text, because it accepts a `std::string`, a string literal or a slice of either without copying. The danger is lifetime: a `string_view` must never outlive the characters it points to, so do not return one that refers to a local string or store one in a long-lived object.

Multidimensional data

A `std::vector<std::vector<double>>` is easy to write but stores each row in a separate allocation. For performance-sensitive grids and matrices, store all elements in a single `std::vector<double>` of size `rows * cols` and compute the index as `r * cols + c`. C++23 adds `std::mdspan`, a non-owning multidimensional view over contiguous data that does this arithmetic for you.

Chapter 7

Pointers and References

Pointers and references let code refer to an object without copying it. They are central to C++: they enable sharing, polymorphism and efficient parameter passing. They are also the source of the language's most serious bugs when lifetimes are misunderstood.

Pointers

```
int value = 42;
int* p = &value;     // p holds the address of value
std::cout << *p;     // dereference: prints 42
*p = 7;              // value is now 7

int* q = nullptr;    // points to nothing
if (q) { /* never runs */ }

struct Point { int x, y; };
Point pt{1, 2};
Point* pp = &pt;
pp->x = 10;          // same as (*pp).x = 10
```

A pointer is a variable whose value is a memory address. `&` takes the address of an object, `*` dereferences a pointer to reach the object, and `->` accesses a member through a pointer. A pointer can be reassigned to point elsewhere, and it can be null. Dereferencing a null or dangling pointer is undefined behaviour, which may crash, silently corrupt data, or appear to work until it doesn't. Always use `nullptr` rather than `0` or `NULL` for null pointers.

Pointer arithmetic

Adding an integer to a pointer moves it by that many elements, not bytes, so `p + 1` points to the next `int` in an array. Pointer arithmetic is only valid within a single array (plus one past the end). It is mostly needed when interfacing with C libraries; modern code uses iterators, `std::span` or indices instead.

References

```
int a = 5;
int& r = a;   // r is another name for a
r = 9;        // a is now 9

void swap_ints(int& x, int& y) {
    int tmp = x;
    x = y;
    y = tmp;
}
```

A reference is an alias for an existing object. It must be initialised when declared, it cannot be reseated to refer to something else, and it cannot be null. These restrictions make references safer and more readable than pointers for parameters and return values. Use a reference when an object must exist; use a pointer when "no object" is meaningful or when you need to re-point.

const with pointers and references

```
const int* p1 = &a;       // pointer to const int: *p1 is read-only
int* const p2 = &a;       // const pointer: p2 cannot be re-pointed
const int* const p3 = &a; // both
const std::string& s = get_name();  // read-only alias
```

Read declarations from right to left: `const int*` is a pointer to a const int, `int* const` is a const pointer to an int. Const-correctness, marking everything that should not change as `const`, documents intent and lets the compiler enforce it. A `const&` parameter can also bind to temporaries, which a non-const reference cannot.

Dangling pointers and references

```
int& bad() {
    int local = 1;
    return local;   // dangling: local dies here
}

std::vector<int> v{1, 2, 3};
int& first = v[0];
v.push_back(4);     // may reallocate: 'first' now dangles
```

A pointer or reference dangles when the object it refers to has been destroyed or moved. Common causes are returning references to locals, holding references into containers that reallocate, and capturing locals by reference in a lambda that outlives them. Compilers catch some cases, and AddressSanitizer (`-fsanitize=address`) catches many more at run time.

std::span

C++20's `std::span<T>` is a non-owning view of a contiguous sequence: a pointer plus a size. A function taking `std::span<const int>` accepts a vector, an array or a C array without copying, and still knows how many elements there are. Like `string_view`, it must not outlive the data it views.

Chapter 8

Dynamic Memory and RAII

Objects in C++ live in one of three places. Static objects live for the whole program. Automatic objects, the local variables of a function, live on the stack and are destroyed automatically at the end of their scope. Dynamic objects live on the heap (the free store) and exist until explicitly destroyed. Managing that last category correctly is one of the core skills of C++ programming.

new and delete

```
int* n = new int(5);          // allocate one int
delete n;                     // free it

int* arr = new int[100];      // allocate an array
delete[] arr;                 // must use delete[] for arrays

Widget* w = new Widget("x");
w->run();
delete w;                     // runs ~Widget(), then frees memory
```

`new` allocates memory and constructs an object; `delete` destroys the object and frees the memory. Every `new` must be matched by exactly one `delete`, and every `new[]` by `delete[]`. Forgetting to delete leaks memory; deleting twice, or using the object after deletion, is undefined behaviour. In practice it is very hard to get this right by hand, especially when exceptions or early returns can skip the `delete`.

The problem with manual management

```
void process() {
    Buffer* buf = new Buffer(1024);
    if (!load(buf)) return;        // leak!
    transform(buf);                // if this throws: leak!
    delete buf;
}
```

Every exit path must release every resource. As functions grow, the number of paths multiplies, and the cleanup code becomes tangled and fragile. The same problem applies to all resources, not just memory: files, sockets, mutex locks, database connections and GPU handles all need to be released exactly once.

RAII: Resource Acquisition Is Initialisation

RAII is the central idiom of C++. A resource is acquired in an object's constructor and released in its destructor. Because C++ guarantees that destructors of automatic objects run when they leave scope, whether by reaching the end of a block, returning early, or unwinding from an exception, the resource is always released, exactly once, without any explicit cleanup code.

```
class File {
public:
    explicit File(const char* path) : f_(std::fopen(path, "r")) {
        if (!f_) throw std::runtime_error("cannot open file");
    }
```

```
    ~File() { std::fclose(f_); }
    File(const File&) = delete;             // no copying
    File& operator=(const File&) = delete;
    std::FILE* get() const { return f_; }
private:
    std::FILE* f_;
};

void read_config() {
    File cfg("app.conf");   // acquired
    parse(cfg.get());       // may throw
}                           // always closed here
```

The Standard Library is built on RAII. `std::vector` and `std::string` own their memory, `std::fstream` owns a file handle, `std::lock_guard` owns a mutex lock, and `std::unique_ptr` owns a heap object. When you compose classes from these members, your class becomes RAII-correct automatically with no destructor of your own.

Guidelines

In modern C++, you should almost never write `new` or `delete` directly. Prefer automatic objects. When you need dynamic lifetime or a variable-size collection, use a container or a smart pointer created with `std::make_unique` or `std::make_shared`. Reserve raw owning pointers for the internals of low-level data structures, and even there, wrap them in a class whose destructor does the cleanup. A raw pointer in modern code should mean "I refer to this, but I do not own it".

Chapter 9

Classes and Objects

A class defines a new type by bundling data with the functions that operate on it. Classes let you model concepts from the problem domain, such as a bank account, a matrix or a network connection, and enforce rules about how they may be used.

Defining a class

```
class BankAccount {
public:
    BankAccount(std::string owner, double balance)
        : owner_(std::move(owner)), balance_(balance) {}

    void deposit(double amount) {
        if (amount <= 0) throw std::invalid_argument("bad amount");
        balance_ += amount;
    }
    bool withdraw(double amount) {
        if (amount > balance_) return false;
        balance_ -= amount;
        return true;
    }
    double balance() const { return balance_; }
    const std::string& owner() const { return owner_; }

private:
    std::string owner_;
    double balance_;
};

BankAccount acct("Ada", 100.0);
acct.deposit(50);
std::cout << acct.balance();   // 150
```

Encapsulation and access control

Members under `public:` are accessible to everyone; members under `private:` only to the class's own member functions and friends; `protected:` additionally allows access from derived classes. Keeping data private and exposing operations protects the class invariant, the rule that must always hold, such as "balance is never negative". If the data were public, any code could break that rule and the class could no longer guarantee it.

`struct` and `class` are the same except for the default access: `struct` members are public by default, `class` members private. By convention, `struct` is used for simple aggregates of data with no invariants, and `class` for types with behaviour and invariants.

Member functions and const

A member function marked `const`, such as `balance() const`, promises not to modify the object. Only const member functions can be called on a const object or through a const reference, so mark every function that does not change state as const. Inside a member function, `this` is a pointer to

the current object, though you rarely need to write it explicitly.

Static members

```
class Connection {
public:
    Connection()  { ++open_count; }
    ~Connection() { --open_count; }
    static int count() { return open_count; }
private:
    static inline int open_count = 0;   // one shared copy
};
```

A `static` data member belongs to the class rather than to any object: all instances share one copy. A static member function has no `this` pointer and can be called as `Connection::count()`. C++17's `inline` variables allow static members to be initialised directly in the class definition.

Separating interface and implementation

In larger programs, the class definition with member declarations goes in a header (`bank_account.h`) and the member function bodies go in a source file (`bank_account.cpp`), written with the class qualifier, for example `void BankAccount::deposit(double amount) { ... }`. This keeps headers small, speeds up compilation and hides implementation details from users of the class. Short functions can stay in the header, where they are implicitly inline.

Designing good classes

A good class has a single, clear responsibility and a small public interface that is easy to use correctly and hard to use incorrectly. Prefer making invalid states unrepresentable: validate in the constructor and throw if the arguments are invalid, so that every object that exists is valid. Prefer free (non-member) functions for operations that can be implemented using only the public interface; this keeps the class minimal.

Chapter 10

Constructors, Destructors and the Rule of Zero

Constructors establish an object's invariant when it is created, and the destructor releases its resources when it is destroyed. C++ also defines special member functions for copying and moving objects. Understanding when these are generated, and when you must write them, is essential to writing correct classes.

Constructors

```
class Rectangle {
public:
    Rectangle() = default;                          // default ctor
    Rectangle(double w, double h) : w_(w), h_(h) {
        if (w < 0 || h < 0) throw std::invalid_argument("negative size");
    }
    explicit Rectangle(double side) : Rectangle(side, side) {} // delegating
private:
    double w_ = 1.0;   // default member initialisers
    double h_ = 1.0;
};
```

Initialise members in the member initialiser list (after the colon) rather than by assignment in the body; it constructs each member directly with the right value and is the only way to initialise const and reference members. Members are initialised in the order they are declared in the class, regardless of the order in the list, so keep the two orders consistent. Default member initialisers provide fallback values. A delegating constructor forwards to another constructor of the same class.

Mark single-argument constructors `explicit` unless you really want implicit conversion. Without it, `Rectangle r = 5.0;` would silently compile, which is rarely intended.

Destructors

The destructor, written `~ClassName()`, runs automatically when an object's lifetime ends. Members are destroyed in the reverse order of their declaration, after the destructor body has run. Destructors must not throw exceptions; they are implicitly `noexcept`, and a throw from a destructor during stack unwinding terminates the program.

The special member functions

Function	Signature
Default constructor	T()
Destructor	~T()
Copy constructor	T(const T&)
Copy assignment	T& operator=(const T&)
Move constructor	T(T&&) noexcept

Move assignment	T& operator=(T&&) noexcept

The compiler generates these automatically when needed, performing member-wise copy, move and destruction. For classes whose members are all well-behaved types, such as numbers, strings, vectors and smart pointers, the generated versions are exactly right.

The Rule of Three, Five and Zero

The Rule of Three says that if a class needs a custom destructor, copy constructor or copy assignment operator, it almost certainly needs all three, because it is managing a resource directly. The Rule of Five extends this to the two move operations introduced in C++11. The Rule of Zero, the modern ideal, says that most classes should declare none of them: hold resources in members that already manage themselves, and let the compiler generate everything.

```
// Rule of Zero: no special members needed
class Document {
    std::string title_;
    std::vector<std::string> lines_;
    std::unique_ptr<Renderer> renderer_;   // move-only => Document is move-only
};
```

Use `= default` to request the compiler-generated version explicitly and `= delete` to forbid an operation, for example to make a class non-copyable. If you write a custom resource-owning class, such as the `File` class in Chapter 8, implement or delete all five operations deliberately; that class then becomes a building block that other classes can use with the Rule of Zero.

Chapter 11

Inheritance and Polymorphism

Inheritance lets a class extend another, reusing its interface and implementation. Combined with virtual functions, it enables run-time polymorphism: code written against a base-class interface can work with any derived class, choosing the right behaviour at run time.

Base and derived classes

```
class Shape {
public:
    virtual ~Shape() = default;              // essential!
    virtual double area() const = 0;         // pure virtual
    virtual std::string name() const { return "shape"; }
};

class Circle : public Shape {
public:
    explicit Circle(double r) : r_(r) {}
    double area() const override { return 3.14159265 * r_ * r_; }
    std::string name() const override { return "circle"; }
private:
    double r_;
};

class Square : public Shape {
public:
    explicit Square(double s) : s_(s) {}
    double area() const override { return s_ * s_; }
private:
    double s_;
};
```

`Circle : public Shape` declares public inheritance, which models an "is-a" relationship: a circle is a shape and can be used wherever a shape is expected. A function declared `= 0` is pure virtual, which makes `Shape` an abstract class that cannot be instantiated; derived classes must override it. Mark overriding functions with `override` so the compiler reports an error if the signature does not actually match a base function.

Polymorphism in action

```
std::vector<std::unique_ptr<Shape>> shapes;
shapes.push_back(std::make_unique<Circle>(1.0));
shapes.push_back(std::make_unique<Square>(2.0));

double total = 0;
for (const auto& s : shapes) {
    total += s->area();          // dynamic dispatch
    std::cout << s->name() << '\n';
}
```

Calls to virtual functions through a pointer or reference are dispatched at run time according to the object's actual type. This is typically implemented with a virtual table: each polymorphic class has a

table of function pointers, and each object holds a hidden pointer to its class's table. The cost is one indirect call and usually prevents inlining, which is negligible in most code but matters in tight loops.

Virtual destructors

If objects will be deleted through a base-class pointer, the base class must have a virtual destructor. Otherwise only the base part is destroyed, which is undefined behaviour and typically leaks the derived part's resources. The rule of thumb: any class with virtual functions should have a public virtual destructor, or a protected non-virtual one if deletion through the base is not allowed.

Object slicing

Copying a derived object into a base-class variable by value, as in `Shape s = circle;`, slices off the derived part, leaving only the base subobject, and virtual calls on the copy use the base implementation. Polymorphic objects should be handled through references or (smart) pointers, never by value. Abstract base classes prevent the mistake entirely, since they cannot be instantiated.

When not to use inheritance

Inheritance creates tight coupling between classes. Prefer composition, containing an object as a member, when you only want to reuse implementation, and reserve public inheritance for genuine substitutability. Keep hierarchies shallow. When the set of types is closed and known in advance, `std::variant` with `std::visit` offers polymorphism without heap allocation or virtual calls. For compile-time polymorphism, templates and concepts are often a better fit.

Mark a class `final` to prevent further derivation, and a virtual function `final` to prevent further overriding; this documents intent and can let the compiler devirtualise calls.

Chapter 12

Operator Overloading

C++ lets user-defined types work with built-in operators. A well-designed `Vector2` can be added with `+`, a `BigInteger` can be compared with `<`, and any type can be printed with `<<`. Used with restraint, operator overloading makes code read like the mathematics or domain it models.

Arithmetic operators

```
struct Vec2 {
    double x{}, y{};

    Vec2& operator+=(const Vec2& o) { x += o.x; y += o.y; return *this; }
    Vec2& operator*=(double k)      { x *= k;  y *= k;  return *this; }
};

// Binary operators as non-members, implemented via compound ones
Vec2 operator+(Vec2 a, const Vec2& b) { return a += b; }
Vec2 operator*(Vec2 v, double k)      { return v *= k; }
Vec2 operator*(double k, Vec2 v)      { return v *= k; }

Vec2 p{1, 2}, q{3, 4};
Vec2 r = p + q * 2.0;   // {7, 10}
```

A common pattern is to implement the compound assignment operator (`+=`) as a member that modifies the object and returns `*this`, then implement the binary operator (`+`) as a non-member that takes its left operand by value and reuses `+=`. Non-member binary operators treat both operands symmetrically, which allows conversions on either side, as with `2.0 * v` and `v * 2.0`.

Comparison operators

```
struct Version {
    int major, minor, patch;
    auto operator<=>(const Version&) const = default;  // C++20
};

Version a{1, 4, 2}, b{1, 10, 0};
bool older = a < b;    // true: compares members in order
bool same  = a == b;   // false
```

Before C++20, a fully comparable type needed six operators. Now, defaulting the three-way comparison operator `<=>` generates `<`, `<=`, `>` and `>=` by comparing members lexicographically in declaration order, and a defaulted `<=>` also implies a defaulted `==` and `!=`. Write a custom `<=>` only when the natural ordering differs from member-wise comparison.

Stream operators

```
std::ostream& operator<<(std::ostream& os, const Vec2& v) {
    return os << '(' << v.x << ", " << v.y << ')';
}

std::cout << Vec2{1.5, 2} << '\n';   // prints (1.5, 2)
```

Output operators must be non-members because the left operand is the stream. They take the stream by reference and return it, which allows chaining. If the operator needs private data, declare it a `friend` inside the class. C++20's `std::format` can also be extended for your types by specialising `std::formatter`.

Other overloadable operators

The subscript operator `operator[]` provides array-like access and is usually written in a const and non-const pair. The function-call operator `operator()` turns objects into function objects, or functors, which is how lambdas are implemented under the hood. `operator->` and unary `operator*` make smart-pointer and iterator types work naturally. Conversion operators such as `explicit operator bool() const` let an object be tested in a condition.

Guidelines

Overload an operator only when its meaning is obvious and conventional for the type. `+` should add, `==` should test equality, and operators should behave like their built-in counterparts: `a + b` should not modify `a`, and `a == b` should imply `!(a != b)`. Never overload `&&`, `||` or the comma operator, because the overloaded versions lose short-circuit evaluation and sequencing guarantees. If a reader would have to look up what an operator does, use a named function instead.

Chapter 13

Templates and Generic Programming

Templates let you write code once and use it with many types. The compiler generates a separate, fully type-checked and optimised version for each type you use, a process called instantiation. Templates are the foundation of the Standard Library: every container and algorithm is a template.

Function templates

```
template <typename T>
T max_of(const T& a, const T& b) {
    return (a < b) ? b : a;
}

max_of(3, 7);                     // T deduced as int
max_of(2.5, 1.5);                 // T = double
max_of<std::string>("ab", "cd");  // explicit
```

The compiler deduces template arguments from the call where it can. The template is checked only when instantiated: `max_of` works with any type that supports `<` and copying. Because the compiler needs the full definition to instantiate it, template code is normally written entirely in header files.

Class templates

```
template <typename T, std::size_t Capacity>
class RingBuffer {
public:
    void push(const T& v) {
        data_[head_] = v;
        head_ = (head_ + 1) % Capacity;
        if (size_ < Capacity) ++size_;
    }
    std::size_t size() const { return size_; }
private:
    std::array<T, Capacity> data_{};
    std::size_t head_ = 0, size_ = 0;
};

RingBuffer<double, 64> samples;
```

Class templates can take type parameters and non-type parameters such as integers. Since C++17, class template argument deduction lets you write `std::vector v{1, 2, 3};` and have the element type deduced. Templates can be specialised: a full or partial specialisation provides a different implementation for particular arguments, as the Standard Library does for `std::vector<bool>`.

Concepts (C++20)

```
#include <concepts>

template <typename T>
concept Numeric = std::integral<T> || std::floating_point<T>;

template <Numeric T>
```

```
T sum(const std::vector<T>& v) {
    T total{};
    for (const T& x : v) total += x;
    return total;
}

// Abbreviated syntax
auto twice(std::integral auto x) { return x * 2; }
```

Without constraints, misusing a template produces long, cryptic error messages from deep inside its implementation. Concepts name the requirements a type must satisfy and check them at the call site, so errors are short and clear. The standard header `<concepts>` provides many ready-made concepts, such as `std::integral`, `std::copyable` and `std::invocable`, and you can define your own with `requires` expressions.

Variadic templates

```
template <typename... Args>
void print_all(const Args&... args) {
    ((std::cout << args << ' '), ...);   // fold expression (C++17)
    std::cout << '\n';
}

print_all(1, "two", 3.0);   // 1 two 3
```

A parameter pack accepts any number of template arguments. Fold expressions apply an operator across all of them. Variadic templates power `std::make_unique`, `std::tuple`, `emplace_back` and `std::format`, all of which accept arbitrary argument lists while preserving type safety.

Compile-time programming

`if constexpr` selects a branch at compile time, discarding the other, which lets one template behave differently for different types without specialisation. Together with `constexpr` functions, type traits from `<type_traits>` and concepts, it allows significant computation to move from run time to compile time. Use this power judiciously: heavy template code increases compile times and can be hard to read.

Chapter 14

STL Containers

The Standard Template Library provides a family of containers, each with different performance characteristics. Choosing the right one is one of the simplest ways to make a program both correct and fast.

Sequence containers

Container	Strengths	Weaknesses
vector	Contiguous, fast iteration, O(1) push_back	Middle insert/erase O(n)
array	Fixed size, no allocation	Size fixed at compile time
deque	O(1) push at both ends	Not contiguous
list	O(1) insert/erase anywhere with iterator	Poor cache use, no indexing
forward_list	Minimal singly linked list	Forward iteration only

Default to `std::vector`. Even for workloads with some insertion in the middle, its cache-friendly layout often beats linked lists in practice. Use `std::deque` when you need efficient insertion at both ends, such as a work queue, and `std::list` only when you need stable iterators and frequent splicing.

Associative containers

```
#include <map>
#include <unordered_map>
#include <set>

std::map<std::string, int> ages{{"Ada", 36}, {"Alan", 41}};
ages["Grace"] = 85;                  // insert or overwrite
if (auto it = ages.find("Alan"); it != ages.end())
    std::cout << it->second;
for (const auto& [name, age] : ages)  // sorted by key
    std::cout << name << ": " << age << '\n';

std::unordered_map<std::string, int> word_count;
for (const auto& w : words) ++word_count[w];  // average O(1)

std::set<int> unique_ids{5, 3, 5, 1};          // {1, 3, 5}
bool has = unique_ids.contains(3);             // C++20
```

`std::map` and `std::set` keep their elements sorted, typically using a red-black tree, giving logarithmic lookup and ordered iteration. `std::unordered_map` and `std::unordered_set` use hash tables, giving average constant-time lookup but no ordering. Prefer the unordered versions for pure lookup unless you need ordering or range queries such as `lower_bound`. The multi- variants allow duplicate keys.

Beware that `map[key]` inserts a default-constructed value if the key is missing. Use `find`, `contains` or `at` when you only want to read. To use a custom type as a key in an unordered container, provide a hash function; for ordered containers, provide `operator<` or `<=>`.

Container adaptors

```
std::stack<int> st;          st.push(1); st.top(); st.pop();
std::queue<int> q;           q.push(1);  q.front(); q.pop();
std::priority_queue<int> pq; pq.push(5); pq.top();  // largest first

// Min-heap of pairs (distance, node), as in Dijkstra's algorithm
std::priority_queue<std::pair<int,int>,
                    std::vector<std::pair<int,int>>,
                    std::greater<>> min_heap;
```

Adaptors provide a restricted interface on top of another container: `stack` (LIFO), `queue` (FIFO) and `priority_queue` (a binary heap). They are convenient for algorithms such as depth-first search, breadth-first search and Dijkstra's shortest path.

Complexity summary

Index access: O(1) for vector, array and deque. Search by key: O(log n) for map and set, O(1) average for unordered_map and unordered_set, O(n) for an unsorted vector. Insertion at the end: amortised O(1) for vector. These guarantees are part of the standard, so you can reason about performance without reading the implementation. Remember that big-O hides constants: for small collections of up to a few dozen elements, a linear search through a vector is frequently the fastest option.

Chapter 15

Iterators, Algorithms and Lambdas

The STL separates data structures from algorithms. Containers expose iterators, and algorithms operate on ranges described by iterators. This design means about a hundred algorithms work with every container, including your own, without duplication.

Iterators

An iterator is a generalised pointer: it refers to an element, can be dereferenced with `*`, and can be advanced with `++`. Every container provides `begin()`, pointing to the first element, and `end()`, pointing one past the last. The half-open range `[begin, end)` is the universal way to describe a sequence. Iterators come in categories, from input iterators that can only move forward once up to random-access and contiguous iterators that support jumping by any distance, and each algorithm states the category it requires.

Lambdas

```
auto square = [](int x) { return x * x; };
square(4);  // 16

int threshold = 10;
auto above = [threshold](int x) { return x > threshold; };  // capture by value
int hits = 0;
auto count_hit = [&hits](int) { ++hits; };                    // capture by reference
auto generic = [](const auto& a, const auto& b) { return a < b; }; // C++14
```

A lambda is an anonymous function object written inline. The capture list in square brackets states which local variables it may use: `[x]` copies, `[&x]` references, `[=]` and `[&]` capture everything used by copy or by reference, and `[this]` captures the current object. Capturing by reference is efficient but dangerous if the lambda outlives the variables, for example when stored or passed to another thread.

Classic algorithms

```
#include <algorithm>
#include <numeric>

std::vector<int> v{5, 2, 8, 1, 9, 3};

std::sort(v.begin(), v.end());                          // 1 2 3 5 8 9
std::sort(v.begin(), v.end(), std::greater<>{});        // descending
auto it = std::find(v.begin(), v.end(), 8);
bool any_big = std::any_of(v.begin(), v.end(), [](int x){ return x > 7; });
int n_even = std::count_if(v.begin(), v.end(), [](int x){ return x % 2 == 0; });
int total = std::accumulate(v.begin(), v.end(), 0);

std::vector<int> squares(v.size());
std::transform(v.begin(), v.end(), squares.begin(), [](int x){ return x * x; });

// erase-remove, simplified in C++20:
std::erase_if(v, [](int x){ return x < 3; });
```

Prefer named algorithms to hand-written loops: `std::any_of` says what it does more clearly than a loop with a flag, and it is equally fast. Sorting is O(n log n); `std::stable_sort` preserves the order of equal elements; `std::binary_search`, `lower_bound` and `upper_bound` find elements in sorted ranges in O(log n). Many algorithms accept an execution policy such as `std::execution::par` to run in parallel.

Ranges (C++20)

```
#include <ranges>
namespace rv = std::views;

std::ranges::sort(v);   // no begin/end needed

auto result = v
    | rv::filter([](int x) { return x % 2 == 0; })
    | rv::transform([](int x) { return x * x; })
    | rv::take(3);

for (int x : result) std::cout << x << ' ';
```

The ranges library lets algorithms take whole containers and adds views: lazy, composable adaptors connected with the pipe operator. A pipeline like the one above allocates nothing and processes elements only as they are iterated. Views are cheap to copy, but like `string_view` they refer to the underlying data, which must outlive them. C++23 adds `std::ranges::to` to collect a view into a container.

Chapter 16

Smart Pointers and Move Semantics

Smart pointers are RAII classes that own a heap object and delete it automatically. Move semantics, introduced in C++11, allow ownership of resources to be transferred cheaply instead of copied. Together they are the foundation of safe, efficient resource management in modern C++.

std::unique_ptr

```
#include <memory>

auto w = std::make_unique<Widget>("main");  // sole owner
w->draw();
std::unique_ptr<Widget> w2 = std::move(w);  // ownership transferred
// w is now null; w2 deletes the Widget when it goes out of scope

std::unique_ptr<int[]> buf = std::make_unique<int[]>(1024);
```

`std::unique_ptr` represents exclusive ownership. It cannot be copied, only moved, and it has no overhead compared with a raw pointer. It is the default choice whenever you need a heap-allocated object: for polymorphic objects, for large objects, or for types that must have a stable address. Pass it by value to a function to transfer ownership; pass a raw pointer or reference when the function merely uses the object.

std::shared_ptr and std::weak_ptr

```
auto cfg = std::make_shared<Config>();
auto a = cfg;          // reference count: 2
{
    auto b = cfg;      // 3
}                      // back to 2
std::weak_ptr<Config> observer = cfg;   // does not keep it alive
if (auto locked = observer.lock()) {    // non-null if still alive
    locked->reload();
}
```

`std::shared_ptr` implements shared ownership with reference counting: the object is destroyed when the last `shared_ptr` to it goes away. The count is updated atomically, which costs something, so use shared ownership only when lifetime genuinely cannot be tied to a single owner. Reference cycles, such as two objects holding `shared_ptr`s to each other, are never freed; break them with `std::weak_ptr`, a non-owning observer that can check whether the object still exists. Always create shared pointers with `std::make_shared`, which allocates the object and the count together.

Lvalues, rvalues and move semantics

An lvalue is an expression with an identity, such as a named variable. An rvalue is a temporary, such as the result of `a + b` or a function returning by value. Since a temporary is about to be destroyed, its resources can be stolen rather than copied. An rvalue reference, written `T&&`, binds to rvalues, and a move constructor `T(T&&)` uses it to take over the source's internals, typically by

copying a pointer and setting the source's pointer to null.

```
class Buffer {
public:
    explicit Buffer(std::size_t n) : size_(n), data_(new char[n]) {}
    ~Buffer() { delete[] data_; }

    Buffer(Buffer&& other) noexcept
        : size_(std::exchange(other.size_, 0)),
          data_(std::exchange(other.data_, nullptr)) {}

    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            delete[] data_;
            size_ = std::exchange(other.size_, 0);
            data_ = std::exchange(other.data_, nullptr);
        }
        return *this;
    }
    Buffer(const Buffer&) = delete;
    Buffer& operator=(const Buffer&) = delete;
private:
    std::size_t size_;
    char* data_;
};
```

std::move and noexcept

`std::move` does not move anything; it is a cast that turns an lvalue into an rvalue, allowing a move to happen. After moving from an object, it remains valid but in an unspecified state, so only assign to it or destroy it. Mark move operations `noexcept`: `std::vector` uses moves when growing only if they cannot throw, and otherwise falls back to slower copies. Do not write `return std::move(local);`, which disables return value optimisation.

Chapter 17

Exceptions and Error Handling

Every non-trivial program must deal with failure: files that do not exist, invalid input, network errors, exhausted memory. C++ offers several mechanisms, and choosing the right one for each situation is part of good design.

Exceptions

```
#include <stdexcept>

double safe_divide(double a, double b) {
    if (b == 0.0) throw std::invalid_argument("division by zero");
    return a / b;
}

try {
    double r = safe_divide(10, 0);
} catch (const std::invalid_argument& e) {
    std::cerr << "Invalid: " << e.what() << '\n';
} catch (const std::exception& e) {
    std::cerr << "Error: " << e.what() << '\n';
}
```

`throw` raises an exception, and control jumps to the nearest matching `catch` handler up the call stack. During this stack unwinding, destructors of all automatic objects are run, which is why RAII and exceptions work so well together. Throw by value and catch by const reference. Handlers are tried in order, so put more specific types first. Derive custom exceptions from `std::runtime_error` or `std::logic_error` so callers can catch them as `std::exception`.

Exception safety guarantees

Functions should document what happens if an exception occurs. The basic guarantee means no resources leak and objects remain in a valid, though possibly changed, state. The strong guarantee means the operation either succeeds completely or has no effect, like a transaction. The no-throw guarantee, expressed with `noexcept`, means the function never throws. Destructors, move operations and swap functions should be `noexcept`. The copy-and-swap idiom, which builds a new state on the side and then swaps it in, is a simple way to achieve the strong guarantee.

Error codes, std::optional and std::expected

```
#include <optional>
#include <expected>   // C++23

std::optional<int> parse_port(std::string_view s) {
    int v{};
    auto [ptr, ec] = std::from_chars(s.data(), s.data() + s.size(), v);
    if (ec != std::errc{} || v <= 0 || v > 65535) return std::nullopt;
    return v;
}

enum class LoadError { NotFound, BadFormat };
```

```
std::expected<Config, LoadError> load_config(const std::string& path) {
    if (!exists(path)) return std::unexpected(LoadError::NotFound);
    return Config{/* ... */};
}

if (auto cfg = load_config("app.toml")) use(*cfg);
else report(cfg.error());
```

Not every failure is exceptional. When absence is a normal outcome, such as a lookup that finds nothing or input that does not parse, return `std::optional<T>`. When the caller needs to know why an operation failed, C++23's `std::expected<T, E>` holds either a value or an error, making failure explicit in the type without the cost of throwing. Many teams use exceptions for rare, unrecoverable conditions and `expected` for anticipated failures.

Assertions

Programming errors, such as violated preconditions or impossible states, are bugs rather than runtime conditions. Check them with `assert` from `<cassert>`, which stops the program in debug builds and compiles to nothing in release builds, or with `static_assert` for conditions that can be checked at compile time. Do not use exceptions to report bugs that should be fixed.

Guidelines

Never ignore errors silently. Mark functions whose return values must be checked with `[[nodiscard]]`. Catch exceptions only where you can actually handle them, typically near the top of a request or at a module boundary. In code compiled with exceptions disabled, common in games and embedded systems, rely on `expected`, error codes and assertions instead.

Chapter 18

Concurrency and Multithreading

Modern processors have many cores, and using them requires running work in parallel. C++11 introduced a formal memory model and a portable threading library, and later standards added higher-level tools. Concurrency is powerful but subtle: the most important skill is avoiding data races.

Starting threads

```
#include <thread>

void worker(int id) {
    std::cout << "worker " << id << '\n';
}

std::thread t1(worker, 1);
std::thread t2([] { worker(2); });
t1.join();   // wait for completion
t2.join();

// C++20: std::jthread joins automatically and supports cancellation
std::jthread t3([](std::stop_token st) {
    while (!st.stop_requested()) { do_work(); }
});
```

A `std::thread` starts running immediately. Before it is destroyed it must be joined, waiting for it to finish, or detached; destroying a joinable thread terminates the program. Prefer `std::jthread`, which joins in its destructor, following RAII, and provides a `stop_token` for cooperative cancellation.

Data races and mutexes

```
#include <mutex>

class Counter {
public:
    void increment() {
        std::lock_guard lock(mu_);   // locks now, unlocks at scope end
        ++value_;
    }
    int get() const {
        std::lock_guard lock(mu_);
        return value_;
    }
private:
    mutable std::mutex mu_;
    int value_ = 0;
};
```

A data race occurs when two threads access the same memory location without synchronisation and at least one of them writes. Data races are undefined behaviour, not merely wrong results. A `std::mutex` ensures only one thread at a time executes a critical section. Always lock through an

RAII guard such as `std::lock_guard`, `std::unique_lock` or `std::scoped_lock`, never by calling `lock()` and `unlock()` manually. When several mutexes must be held at once, lock them together with `std::scoped_lock` to avoid deadlock.

Atomics

For simple shared counters and flags, `std::atomic<int>` or `std::atomic<bool>` provide lock-free, race-free operations such as `fetch_add`, `load` and `store`. Atomics default to sequentially consistent ordering, the easiest to reason about. Weaker memory orderings exist for experts optimising lock-free data structures, but they are easy to get wrong.

Condition variables

A `std::condition_variable` lets threads wait efficiently until a condition becomes true, such as a queue becoming non-empty, without busy-waiting. A waiting thread holds a `std::unique_lock`, calls `cv.wait(lock, predicate)`, and is woken when another thread calls `notify_one()` or `notify_all()` after changing the shared state. Always wait with a predicate, because spurious wake-ups can occur.

Tasks and futures

```
#include <future>

std::future<long> f = std::async(std::launch::async, [] {
    return expensive_computation();
});
// ... do other work ...
long result = f.get();   // waits if not finished; rethrows exceptions
```

`std::async` runs a function asynchronously and returns a `std::future` from which the result can later be retrieved, including any exception the task threw. For data-parallel work, the parallel algorithms (`std::sort(std::execution::par, ...)`) are often the simplest option. C++20 also adds semaphores, latches and barriers, and coroutines for asynchronous code that reads sequentially.

Design to share as little mutable state as possible. Pass data to threads by value, communicate through thread-safe queues, and make shared data immutable where you can. Use ThreadSanitizer (`-fsanitize=thread`) to detect data races during testing.

Chapter 19

Modern C++ Features and Best Practices

The last decade has transformed C++. Code written in the C++20/23 style is safer, shorter and often faster than the C++98 style many older tutorials teach. This final chapter surveys key modern features and summarises the practices that distinguish good C++ code.

Structured bindings, optional and variant

```
auto [it, inserted] = scores.insert({"Ada", 99});   // C++17
for (const auto& [key, value] : scores) { /* ... */ }

std::variant<int, double, std::string> v = 3.14;
std::visit([](const auto& x) { std::cout << x; }, v);
```

Structured bindings unpack pairs, tuples, arrays and simple structs into named variables. `std::optional<T>` represents a value that may be absent. `std::variant` is a type-safe union holding exactly one of several types, inspected with `std::visit`. `std::any` holds any copyable type but is rarely the best choice.

Formatting and printing

```
#include <format>
#include <print>     // C++23

std::string s = std::format("{} scored {:.1f}%", name, pct);
std::println("Total: {:>8}", total);   // right-aligned, width 8
```

`std::format` provides type-safe, Python-like formatting, replacing error-prone `printf` and verbose stream manipulators. C++23's `std::print` and `std::println` write formatted output directly and are faster than `std::cout`.

Modules and coroutines

C++20 modules replace the textual inclusion of headers. A module is compiled once and imported with `import`, which speeds up builds and prevents macros from leaking between components; C++23 adds `import std;` for the entire Standard Library. Coroutines are functions that can suspend and resume, using `co_await`, `co_yield` and `co_return`. They enable generators, such as C++23's `std::generator`, and asynchronous code that reads like ordinary sequential code.

Best practices checklist

Manage resources with RAII; never use raw `new` and `delete` in application code. Prefer `std::vector`, `std::string` and the other standard containers to hand-rolled data structures. Use `std::unique_ptr` by default and `std::shared_ptr` only for genuinely shared ownership. Follow the Rule of Zero. Make everything `const` that can be, and pass large read-only parameters by `const&`, `std::string_view` or `std::span`. Initialise every variable, preferably with braces. Use `enum class`, `nullptr`, `override` and `explicit`.

Prefer standard algorithms and ranges to raw loops, and named functions to clever code. Constrain templates with concepts. Keep functions short and classes focused on a single responsibility. Use `std::optional` and `std::expected` for expected failures and exceptions for exceptional ones.

Tools that help

Compile with high warning levels (`-Wall -Wextra -Wpedantic`) and fix every warning. Run tests under AddressSanitizer, UndefinedBehaviorSanitizer and ThreadSanitizer. Use clang-tidy and the C++ Core Guidelines checks for static analysis, clang-format for consistent style, and a test framework such as GoogleTest or Catch2. Profile before optimising: tools like perf, VTune and Tracy show where time is actually spent.

Where to go next

The C++ Core Guidelines, maintained by Bjarne Stroustrup and Herb Sutter, are the most authoritative free reference for modern style. cppreference.com documents every language feature and library component. Stroustrup's A Tour of C++ is a concise introduction to the modern language, and Scott Meyers' Effective Modern C++ explains the subtleties of C++11 and C++14. The best way to learn, however, is to build things: write small programs, read good open-source code, and keep your compiler's warnings turned on.